SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 3- CODING CHALLENGE QUESTIONS

ANNEXURE A

CODING CHALLENGE QUESTIONS

Course Code: CSA12	**Course Title: Computer Architecture**
CO Assessed: CO1	**Assessment: ASSESSMENT TOOL3- CODING CHALLENGE QUESTIONS**
Weightage: 40%	**Total Marks: 25**
CO1 Statement: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)	
Student Name: G.SARANRAJ	**Reg.No.:192511182**
Department: BE . CSE	**Date: 23/07/2026**

1. Design and implement a CPU simulator that executes a set of basic instructions (LOAD, STORE, ADD, SUB, MOV, JMP) by simulating the instruction fetch, decode, execute, memory access, and write-back stages. Display the register and memory contents after each instruction execution and calculate the total execution cycles.

Answer (Python implementation):

```
"""
Q1. CPU Simulator - LOAD, STORE, ADD, SUB, MOV, JMP
Simulates Fetch -> Decode -> Execute -> Memory Access -> Write-Back stages.
Displays registers and memory after every instruction and reports total
execution cycles (each stage = 1 cycle).
"""

CYCLES_PER_STAGE = 1
STAGES = ["FETCH", "DECODE", "EXECUTE", "MEMORY", "WRITEBACK"]


class CPUSimulator:
    def __init__(self, memory_size=32):
        self.registers = {"R0": 0, "R1": 0, "R2": 0, "R3": 0}
        self.memory = [0] * memory_size
        self.pc = 0
        self.program = []
        self.total_cycles = 0

    def load_program(self, program):
        """program: list of instruction strings, e.g. 'LOAD R0 5'"""
        self.program = program
        self.pc = 0

    def run(self):
        while self.pc < len(self.program):
            instr = self.program[self.pc]
```

```
            self.execute_instruction(instr)

    def execute_instruction(self, instr):
        parts = instr.replace(",", " ").split()
        opcode = parts[0].upper()

        # ---- FETCH ----
        print(f"\n[PC={self.pc}] Instruction Fetched: {instr}")
        self.total_cycles += CYCLES_PER_STAGE

        # ---- DECODE ----
        operands = parts[1:]
        print(f"  DECODE : opcode={opcode}, operands={operands}")
        self.total_cycles += CYCLES_PER_STAGE

        # ---- EXECUTE / MEMORY / WRITEBACK ----
        jumped = False
        if opcode == "LOAD":                        # LOAD Rd, addr
            reg, addr = operands[0], int(operands[1])
            self.total_cycles += CYCLES_PER_STAGE   # EXECUTE (address calc)
            value = self.memory[addr]               # MEMORY
            self.total_cycles += CYCLES_PER_STAGE
            self.registers[reg] = value              # WRITEBACK
            self.total_cycles += CYCLES_PER_STAGE

        elif opcode == "STORE":                     # STORE Rs, addr
            reg, addr = operands[0], int(operands[1])
            self.total_cycles += CYCLES_PER_STAGE   # EXECUTE
            self.memory[addr] = self.registers[reg]  # MEMORY (write)
            self.total_cycles += 2 * CYCLES_PER_STAGE  # MEMORY +
WRITEBACK(none, but stage still counted)

        elif opcode == "ADD":                       # ADD Rd, Rs1, Rs2
            rd, r1, r2 = operands
            self.total_cycles += CYCLES_PER_STAGE   # EXECUTE (ALU op)
            result = self.registers[r1] + self.registers[r2]
            self.total_cycles += CYCLES_PER_STAGE   # MEMORY (not used)
            self.registers[rd] = result             # WRITEBACK
            self.total_cycles += CYCLES_PER_STAGE

        elif opcode == "SUB":                       # SUB Rd, Rs1, Rs2
            rd, r1, r2 = operands
            self.total_cycles += CYCLES_PER_STAGE
            result = self.registers[r1] - self.registers[r2]
            self.total_cycles += CYCLES_PER_STAGE
            self.registers[rd] = result
            self.total_cycles += CYCLES_PER_STAGE

        elif opcode == "MOV":                       # MOV Rd, Rs  or  MOV Rd,
#imm
```

```
            rd, src = operands
            self.total_cycles += CYCLES_PER_STAGE
            if src.startswith("#"):
                value = int(src[1:])
            else:
                value = self.registers[src]
            self.total_cycles += CYCLES_PER_STAGE
            self.registers[rd] = value
            self.total_cycles += CYCLES_PER_STAGE

        elif opcode == "JMP":                       # JMP target_index
            target = int(operands[0])
            self.total_cycles += 3 * CYCLES_PER_STAGE  #
EXECUTE+MEMORY+WRITEBACK
            self.pc = target
            jumped = True

        else:
            raise ValueError(f"Unknown opcode: {opcode}")

        if not jumped:
            self.pc += 1

        self.display_state()

    def display_state(self):
        print("  Registers :", self.registers)
        print("  Memory[0:8]:", self.memory[:8])


if __name__ == "__main__":
    cpu = CPUSimulator()
    # Pre-load some memory values
    cpu.memory[0] = 10
    cpu.memory[1] = 20

    program = [
        "LOAD R0 0",     # R0 = mem[0] = 10
        "LOAD R1 1",     # R1 = mem[1] = 20
        "ADD R2 R0 R1",  # R2 = R0 + R1 = 30
        "SUB R3 R1 R0",  # R3 = R1 - R0 = 10
        "STORE R2 2",    # mem[2] = R2
        "MOV R0 #5",     # R0 = 5
    ]

    cpu.load_program(program)
    cpu.run()

    print("\n===== FINAL STATE =====")
    cpu.display_state()
```

```
    print(f"Total execution cycles: {cpu.total_cycles}")
```

Sample Output:

```
[PC=0] Instruction Fetched: LOAD R0 0
  DECODE : opcode=LOAD, operands=['R0', '0']
  Registers : {'R0': 10, 'R1': 0, 'R2': 0, 'R3': 0}
  Memory[0:8]: [10, 20, 0, 0, 0, 0, 0, 0]

[PC=1] Instruction Fetched: LOAD R1 1
  DECODE : opcode=LOAD, operands=['R1', '1']
  Registers : {'R0': 10, 'R1': 20, 'R2': 0, 'R3': 0}
  Memory[0:8]: [10, 20, 0, 0, 0, 0, 0, 0]

[PC=2] Instruction Fetched: ADD R2 R0 R1
  DECODE : opcode=ADD, operands=['R2', 'R0', 'R1']
  Registers : {'R0': 10, 'R1': 20, 'R2': 30, 'R3': 0}
  Memory[0:8]: [10, 20, 0, 0, 0, 0, 0, 0]

[PC=3] Instruction Fetched: SUB R3 R1 R0
  DECODE : opcode=SUB, operands=['R3', 'R1', 'R0']
  Registers : {'R0': 10, 'R1': 20, 'R2': 30, 'R3': 10}
  Memory[0:8]: [10, 20, 0, 0, 0, 0, 0, 0]

[PC=4] Instruction Fetched: STORE R2 2
  DECODE : opcode=STORE, operands=['R2', '2']
  Registers : {'R0': 10, 'R1': 20, 'R2': 30, 'R3': 10}
  Memory[0:8]: [10, 20, 30, 0, 0, 0, 0, 0]

[PC=5] Instruction Fetched: MOV R0 #5
  DECODE : opcode=MOV, operands=['R0', '#5']
  Registers : {'R0': 5, 'R1': 20, 'R2': 30, 'R3': 10}
  Memory[0:8]: [10, 20, 30, 0, 0, 0, 0, 0]

===== FINAL STATE =====
  Registers : {'R0': 5, 'R1': 20, 'R2': 30, 'R3': 10}
  Memory[0:8]: [10, 20, 30, 0, 0, 0, 0, 0]
Total execution cycles: 30
```

2. Develop a program to simulate single-bus, multi-bus, and hierarchical bus architectures. Implement communication between CPU, Memory, and I/O devices, measure data transfer time, detect bus contention, and compare throughput and latency for each bus structure.

Answer (Python implementation):

```
"""
Q2. Simulate Single-Bus, Multi-Bus, and Hierarchical Bus architectures.
Models CPU / Memory / I/O communication requests, measures data transfer
time, detects bus contention, and compares throughput & latency.
```

```
"""

import random

random.seed(42)

TRANSFER_TIME = {"CPU-MEM": 2, "CPU-IO": 4}  # base time units per transfer


class Request:
    def __init__(self, source, dest, req_id):
        self.source = source
        self.dest = dest
        self.id = req_id
        self.kind = "CPU-MEM" if dest == "MEM" else "CPU-IO"


def generate_requests(n=8):
    reqs = []
    for i in range(n):
        dest = random.choice(["MEM", "IO"])
        reqs.append(Request("CPU", dest, i))
    return reqs


class SingleBus:
    """One shared bus: only one transfer can use it at a time."""
    name = "Single Bus"

    def simulate(self, requests):
        time_cursor = 0
        contention = 0
        log = []
        for req in requests:
            duration = TRANSFER_TIME[req.kind]
            start = time_cursor          # bus is busy until previous
finishes
            if start > 0:
                contention += 1          # every request after the first
waits
            end = start + duration
            log.append((req.id, req.kind, start, end))
            time_cursor = end
        total_time = time_cursor
        throughput = len(requests) / total_time
        avg_latency = sum(end - start for (_, _, start, end) in log) /
len(log)
        return {
            "bus": self.name,
            "total_time": total_time,
```

```
            "throughput": round(throughput, 3),
            "avg_latency": round(avg_latency, 3),
            "contention_events": contention,
            "log": log,
        }


class MultiBus:
    """Separate memory bus and I/O bus - transfers of different kinds run in
parallel."""
    name = "Multi Bus"

    def simulate(self, requests):
        mem_cursor = 0
        io_cursor = 0
        contention = 0
        log = []
        for req in requests:
            duration = TRANSFER_TIME[req.kind]
            if req.kind == "CPU-MEM":
                start = mem_cursor
                if start > 0:
                    contention += 1 if start < duration else 0
                end = start + duration
                mem_cursor = end
            else:
                start = io_cursor
                if start > 0:
                    contention += 1 if start < duration else 0
                end = start + duration
                io_cursor = end
            log.append((req.id, req.kind, start, end))
        total_time = max(mem_cursor, io_cursor)
        throughput = len(requests) / total_time
        avg_latency = sum(end - start for (_, _, start, end) in log) /
len(log)
        return {
            "bus": self.name,
            "total_time": total_time,
            "throughput": round(throughput, 3),
            "avg_latency": round(avg_latency, 3),
            "contention_events": contention,
            "log": log,
        }


class HierarchicalBus:
    """
    Local (CPU-cache/mem) high-speed bus + a bridged system/IO bus.
    MEM requests use the fast local bus; IO requests cross a bridge onto
```

```
    a slower expansion bus, but both can run concurrently.
    """
    name = "Hierarchical Bus"
    BRIDGE_DELAY = 1

    def simulate(self, requests):
        local_cursor = 0
        io_cursor = 0
        contention = 0
        log = []
        for req in requests:
            if req.kind == "CPU-MEM":
                duration = TRANSFER_TIME[req.kind]
                start = local_cursor
                if start > 0:
                    contention += 0  # local bus is fast/dedicated -
negligible contention
                end = start + duration
                local_cursor = end
            else:
                duration = TRANSFER_TIME[req.kind] + self.BRIDGE_DELAY
                start = io_cursor
                if start > 0:
                    contention += 1 if start < duration else 0
                end = start + duration
                io_cursor = end
            log.append((req.id, req.kind, start, end))
        total_time = max(local_cursor, io_cursor)
        throughput = len(requests) / total_time
        avg_latency = sum(end - start for (_, _, start, end) in log) /
len(log)
        return {
            "bus": self.name,
            "total_time": total_time,
            "throughput": round(throughput, 3),
            "avg_latency": round(avg_latency, 3),
            "contention_events": contention,
            "log": log,
        }


def print_report(result):
    print(f"\n--- {result['bus']} ---")
    for (rid, kind, start, end) in result["log"]:
        print(f"  Req {rid:2d} [{kind}] : start={start}, end={end}")
    print(f"  Total time        : {result['total_time']}")
    print(f"  Throughput (req/t): {result['throughput']}")
    print(f"  Avg latency       : {result['avg_latency']}")
    print(f"  Contention events : {result['contention_events']}")
```

```
if __name__ == "__main__":
    requests = generate_requests(8)
    print("Generated requests:", [(r.id, r.kind) for r in requests])

    results = [SingleBus().simulate(requests),
               MultiBus().simulate(requests),
               HierarchicalBus().simulate(requests)]

    for r in results:
        print_report(r)

    print("\n===== COMPARISON =====")

print(f"{'Bus':<18}{'TotalTime':<12}{'Throughput':<12}{'AvgLatency':<12}{'Con
tention'}")
    for r in results:

print(f"{r['bus']:<18}{r['total_time']:<12}{r['throughput']:<12}{r['avg_laten
cy']:<12}{r['contention_events']}")
```

Sample Output:

```
Generated requests: [(0, 'CPU-MEM'), (1, 'CPU-MEM'), (2, 'CPU-IO'), (3, 'CPU-
MEM'), (4, 'CPU-MEM'), (5, 'CPU-MEM'), (6, 'CPU-MEM'), (7, 'CPU-MEM')]

--- Single Bus ---
  Req  0 [CPU-MEM] : start=0, end=2
  Req  1 [CPU-MEM] : start=2, end=4
  Req  2 [CPU-IO] : start=4, end=8
  Req  3 [CPU-MEM] : start=8, end=10
  Req  4 [CPU-MEM] : start=10, end=12
  Req  5 [CPU-MEM] : start=12, end=14
  Req  6 [CPU-MEM] : start=14, end=16
  Req  7 [CPU-MEM] : start=16, end=18
  Total time        : 18
  Throughput (req/t): 0.444
  Avg latency       : 2.25
  Contention events : 7

--- Multi Bus ---
  Req  0 [CPU-MEM] : start=0, end=2
  Req  1 [CPU-MEM] : start=2, end=4
  Req  2 [CPU-IO] : start=0, end=4
  Req  3 [CPU-MEM] : start=4, end=6
  Req  4 [CPU-MEM] : start=6, end=8
  Req  5 [CPU-MEM] : start=8, end=10
  Req  6 [CPU-MEM] : start=10, end=12
  Req  7 [CPU-MEM] : start=12, end=14
  Total time        : 14
```

```
  Throughput (req/t): 0.571
  Avg latency       : 2.25
  Contention events : 0

--- Hierarchical Bus ---
  Req  0 [CPU-MEM] : start=0, end=2
  Req  1 [CPU-MEM] : start=2, end=4
  Req  2 [CPU-IO] : start=0, end=5
  Req  3 [CPU-MEM] : start=4, end=6
  Req  4 [CPU-MEM] : start=6, end=8
  Req  5 [CPU-MEM] : start=8, end=10
  Req  6 [CPU-MEM] : start=10, end=12
  Req  7 [CPU-MEM] : start=12, end=14
  Total time        : 14
  Throughput (req/t): 0.571
  Avg latency       : 2.375
  Contention events : 0

===== COMPARISON =====
Bus               TotalTime   Throughput  AvgLatency  Contention
Single Bus        18          0.444       2.25        7
Multi Bus         14          0.571       2.25        0
Hierarchical Bus  14          0.571       2.375       0
```

3. Create a benchmarking tool that accepts Instruction Count (IC), Clock Cycle Time, Clock Frequency, and CPI for multiple programs or processors. Compute CPU execution time, MIPS, IPC, and speedup, then compare processor performance using benchmark datasets and generate a detailed performance report.

Answer (Python implementation):

```
"""
Q3. Benchmarking tool.
Accepts Instruction Count (IC), Clock Cycle Time, Clock Frequency and CPI
for multiple programs/processors. Computes CPU execution time, MIPS, IPC
and speedup, then compares processors and prints a performance report.
"""



class Processor:
    def __init__(self, name, instruction_count, clock_frequency_hz, cpi):
        self.name = name
        self.ic = instruction_count
        self.freq = clock_frequency_hz
        self.cpi = cpi
        self.cycle_time = 1 / clock_frequency_hz

    def execution_time(self):
        # Execution time = IC * CPI * Clock cycle time
        return self.ic * self.cpi * self.cycle_time
```

```
    def mips(self):
        # MIPS = IC / (Execution time * 10^6)
        return self.ic / (self.execution_time() * 1e6)

    def ipc(self):
        return 1 / self.cpi

    def report(self):
        return {
            "name": self.name,
            "IC": self.ic,
            "CPI": self.cpi,
            "Frequency(MHz)": self.freq / 1e6,
            "ExecTime(s)": round(self.execution_time(), 6),
            "MIPS": round(self.mips(), 3),
            "IPC": round(self.ipc(), 3),
        }


def compare(processors):
    reports = [p.report() for p in processors]
    baseline = reports[0]
    for r in reports:
        r["Speedup_vs_" + baseline["name"]] = round(
            baseline["ExecTime(s)"] / r["ExecTime(s)"], 3
        )
    return reports


def print_report(reports):
    headers = list(reports[0].keys())
    col_w = 16
    print("".join(h.ljust(col_w) for h in headers))
    for r in reports:
        print("".join(str(r[h]).ljust(col_w) for h in headers))


if __name__ == "__main__":
    processors = [
        Processor("ProcA", instruction_count=2_000_000,
clock_frequency_hz=1e9, cpi=1.5),
        Processor("ProcB", instruction_count=2_000_000,
clock_frequency_hz=1.5e9, cpi=1.2),
        Processor("ProcC", instruction_count=2_000_000,
clock_frequency_hz=2e9, cpi=2.0),
    ]

    reports = compare(processors)
```

```
    print("===== PERFORMANCE REPORT =====")
    print_report(reports)

    fastest = min(reports, key=lambda r: r["ExecTime(s)"])
    print(f"\nFastest processor: {fastest['name']} "
          f"(execution time = {fastest['ExecTime(s)']} s)")
```

Sample Output:

```
===== PERFORMANCE REPORT =====
name            IC              CPI             Frequency(MHz)  ExecTime(s)
MIPS            IPC             Speedup_vs_ProcA
ProcA           2000000         1.5             1000.0          0.003
666.667         0.667           1.0
ProcB           2000000         1.2             1500.0          0.0016
1250.0          0.833           1.875
ProcC           2000000         2.0             2000.0          0.002
1000.0          0.5             1.5

Fastest processor: ProcB (execution time = 0.0016 s)
```

4. Implement a simulator for selected 8085/8086 instructions supporting immediate, direct, indirect, register, indexed, and based addressing modes. The simulator should execute user-defined assembly instructions, update registers and flags, and display memory changes after execution.

Answer (Python implementation):

```
"""
Q4. 8085/8086 instruction simulator supporting immediate, direct, indirect,
register, indexed and based addressing modes. Executes simple user defined
assembly instructions (MOV/ADD/SUB), updates registers & flags, and shows
memory changes.

Supported operand syntax:
  #n        -> immediate           e.g. #25
  [addr]    -> direct               e.g. [40]
  Rn        -> register             e.g. R1
  @Rn       -> register indirect    e.g. @R2   (Rn holds an address)
  [Rn+off]  -> indexed              e.g. [R3+2]
  BASE(Rn)+off -> based addressing  e.g. BASE(R0)+4
"""

import re


class MicroSim:
    def __init__(self, mem_size=64):
        self.registers = {f"R{i}": 0 for i in range(6)}
        self.memory = [0] * mem_size
        self.flags = {"ZF": 0, "SF": 0, "CF": 0}
```

```
    # ---------- addressing-mode resolution ----------
    def resolve_operand(self, token, for_write=False):
        token = token.strip()

        if token.startswith("#"):                    # immediate
            return int(token[1:]), None

        if token.startswith("@"):                     # register indirect
            reg = token[1:]
            addr = self.registers[reg]
            return (self.memory[addr] if not for_write else None), ("mem",
addr)

        m = re.match(r"^\[(R\d+)\+(-?\d+)\]$", token)   # indexed: [Rn+off]
        if m:
            reg, off = m.group(1), int(m.group(2))
            addr = self.registers[reg] + off
            return (self.memory[addr] if not for_write else None), ("mem",
addr)

        m = re.match(r"^BASE\((R\d+)\)\+(-?\d+)$", token)  # based:
BASE(Rn)+off
        if m:
            reg, off = m.group(1), int(m.group(2))
            addr = self.registers[reg] + off
            return (self.memory[addr] if not for_write else None), ("mem",
addr)

        m = re.match(r"^\[(\d+)\]$", token)             # direct: [addr]
        if m:
            addr = int(m.group(1))
            return (self.memory[addr] if not for_write else None), ("mem",
addr)

        if re.match(r"^R\d+$", token):                  # register
            return (self.registers[token] if not for_write else None),
("reg", token)

        raise ValueError(f"Unrecognised operand/addressing mode: {token}")

    def write_back(self, dest, value):
        kind, loc = dest
        if kind == "reg":
            self.registers[loc] = value
        else:
            self.memory[loc] = value
        self.update_flags(value)

    def update_flags(self, value):
```

```
        self.flags["ZF"] = 1 if value == 0 else 0
        self.flags["SF"] = 1 if value < 0 else 0
        self.flags["CF"] = 1 if value > 255 else 0

    # ---------- instruction execution ----------
    def execute(self, instr):
        parts = instr.replace(",", " ").split()
        op = parts[0].upper()
        args = parts[1:]

        if op == "MOV":                                  # MOV dest, src
            dest_tok, src_tok = args
            value, dest = self.resolve_operand(src_tok)
            _, dest_loc = self.resolve_operand(dest_tok, for_write=True)
            self.write_back(dest_loc, value)

        elif op == "ADD":                                 # ADD dest, src
            dest_tok, src_tok = args
            src_val, _ = self.resolve_operand(src_tok)
            dest_val, dest_loc = self.resolve_operand(dest_tok)
            result = dest_val + src_val
            self.write_back(dest_loc, result)

        elif op == "SUB":                                 # SUB dest, src
            dest_tok, src_tok = args
            src_val, _ = self.resolve_operand(src_tok)
            dest_val, dest_loc = self.resolve_operand(dest_tok)
            result = dest_val - src_val
            self.write_back(dest_loc, result)

        else:
            raise ValueError(f"Unsupported instruction: {op}")

        print(f"Executed: {instr:<22} Registers={self.registers}
Flags={self.flags}")

    def run(self, program):
        for instr in program:
            self.execute(instr)


if __name__ == "__main__":
    sim = MicroSim()
    sim.memory[10] = 7
    sim.memory[20] = 3
    sim.registers["R2"] = 10      # R2 used as a pointer for indirect
addressing
    sim.registers["R3"] = 18      # R3 used as base for indexed addressing
(R3+2 -> 20)
```

```
    program = [
        "MOV R0, #25",        # immediate  -> R0 = 25
        "MOV R1, [10]",       # direct     -> R1 = mem[10] = 7
        "MOV R4, @R2",        # indirect   -> R4 = mem[R2] = mem[10] = 7
        "MOV R5, [R3+2]",     # indexed    -> R5 = mem[R3+2] = mem[20] = 3
        "ADD R0, R1",         # register   -> R0 = 25 + 7 = 32
        "SUB R0, #2",         # immediate  -> R0 = 32 - 2 = 30
        "MOV [30], R0",       # direct write -> mem[30] = 30
    ]

    print("Initial memory[10],[20]:", sim.memory[10], sim.memory[20])
    sim.run(program)

    print("\n===== FINAL STATE =====")
    print("Registers:", sim.registers)
    print("Flags    :", sim.flags)
    print("Memory[10,20,30]:", sim.memory[10], sim.memory[20],
sim.memory[30])
```

Sample Output:

```
Initial memory[10],[20]: 7 3
Executed: MOV R0, #25           Registers={'R0': 25, 'R1': 0, 'R2': 10,
'R3': 18, 'R4': 0, 'R5': 0}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}
Executed: MOV R1, [10]          Registers={'R0': 25, 'R1': 7, 'R2': 10,
'R3': 18, 'R4': 0, 'R5': 0}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}
Executed: MOV R4, @R2           Registers={'R0': 25, 'R1': 7, 'R2': 10,
'R3': 18, 'R4': 7, 'R5': 0}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}
Executed: MOV R5, [R3+2]        Registers={'R0': 25, 'R1': 7, 'R2': 10,
'R3': 18, 'R4': 7, 'R5': 3}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}
Executed: ADD R0, R1            Registers={'R0': 32, 'R1': 7, 'R2': 10,
'R3': 18, 'R4': 7, 'R5': 3}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}
Executed: SUB R0, #2            Registers={'R0': 30, 'R1': 7, 'R2': 10,
'R3': 18, 'R4': 7, 'R5': 3}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}
Executed: MOV [30], R0          Registers={'R0': 30, 'R1': 7, 'R2': 10,
'R3': 18, 'R4': 7, 'R5': 3}  Flags={'ZF': 0, 'SF': 0, 'CF': 0}

===== FINAL STATE =====
Registers: {'R0': 30, 'R1': 7, 'R2': 10, 'R3': 18, 'R4': 7, 'R5': 3}
Flags    : {'ZF': 0, 'SF': 0, 'CF': 0}
Memory[10,20,30]: 7 3 30
```

5. Develop an application that converts binary, decimal, hexadecimal, and octal numbers, performs arithmetic operations in different number systems, and encodes simple assembly instructions into machine code based on predefined instruction formats and addressing modes. The program should also decode machine instructions back into assembly language.

Answer (Python implementation):

```
"""
```

```
Q5. Number system converter + simple assembly <-> machine code
encoder/decoder.

Part A: convert between binary, decimal, hexadecimal and octal, and
         perform arithmetic in a chosen base.
Part B: encode a simple assembly instruction into machine code using a
         fixed instruction format, and decode machine code back to assembly.
"""


# ---------------- Part A: number system conversion ----------------
def to_decimal(value: str, base: int) -> int:
    return int(value, base)


def from_decimal(value: int, base: int) -> str:
    if base == 2:
        return bin(value)[2:] if value >= 0 else "-" + bin(value)[3:]
    if base == 8:
        return oct(value)[2:] if value >= 0 else "-" + oct(value)[3:]
    if base == 16:
        return hex(value)[2:].upper() if value >= 0 else "-" +
hex(value)[3:].upper()
    if base == 10:
        return str(value)
    raise ValueError("Unsupported base")


def convert(value: str, from_base: int, to_base: int) -> str:
    dec = to_decimal(value, from_base)
    return from_decimal(dec, to_base)


def arithmetic(a: str, b: str, base: int, op: str) -> str:
    x, y = to_decimal(a, base), to_decimal(b, base)
    result = {"+": x + y, "-": x - y, "*": x * y, "/": x // y if y else
None}[op]
    return from_decimal(result, base)


# ---------------- Part B: instruction encode / decode ----------------
# Fixed 12-bit instruction format:  [ 4-bit opcode | 4-bit reg | 4-bit
operand ]
OPCODES = {"MOV": 0b0001, "ADD": 0b0010, "SUB": 0b0011, "LOAD": 0b0100,
"STORE": 0b0101}
OPCODES_REV = {v: k for k, v in OPCODES.items()}
REGISTERS = {"R0": 0, "R1": 1, "R2": 2, "R3": 3}
REGISTERS_REV = {v: k for k, v in REGISTERS.items()}
```

```
def encode_instruction(mnemonic: str, reg: str, operand: int) -> str:
    """Returns a 12-bit binary string: opcode(4) | reg(4) | operand(4)"""
    opcode = OPCODES[mnemonic.upper()]
    reg_code = REGISTERS[reg.upper()]
    operand &= 0xF  # fit into 4 bits
    machine_code = (opcode << 8) | (reg_code << 4) | operand
    return format(machine_code, "012b")


def decode_instruction(machine_code: str) -> str:
    """Takes a 12-bit binary string and returns the assembly mnemonic
form."""
    code = int(machine_code, 2)
    opcode = (code >> 8) & 0xF
    reg_code = (code >> 4) & 0xF
    operand = code & 0xF
    mnemonic = OPCODES_REV.get(opcode, "UNKNOWN")
    reg = REGISTERS_REV.get(reg_code, f"R?{reg_code}")
    return f"{mnemonic} {reg}, #{operand}"


if __name__ == "__main__":
    print("===== PART A: NUMBER SYSTEM CONVERSION =====")
    print("Decimal 156  -> Binary :", from_decimal(156, 2))
    print("Decimal 156  -> Octal  :", from_decimal(156, 8))
    print("Decimal 156  -> Hex    :", from_decimal(156, 16))
    print("Hex 9C       -> Decimal:", to_decimal("9C", 16))
    print("Binary 10011100 -> Hex :", convert("10011100", 2, 16))

    print("\nArithmetic in hex : 1A + 0F =", arithmetic("1A", "0F", 16, "+"))
    print("Arithmetic in binary: 1010 - 0011 =", arithmetic("1010", "0011",
2, "-"))
    print("Arithmetic in octal: 17 * 3 =", arithmetic("17", "3", 8, "*"))

    print("\n===== PART B: INSTRUCTION ENCODE / DECODE =====")
    instructions = [
        ("MOV", "R0", 5),
        ("ADD", "R1", 9),
        ("SUB", "R2", 3),
        ("LOAD", "R3", 7),
    ]
    machine_codes = []
    for mnemonic, reg, operand in instructions:
        code = encode_instruction(mnemonic, reg, operand)
        machine_codes.append(code)
        print(f"{mnemonic} {reg}, #{operand}  -->  {code}  (hex:
{hex(int(code,2))})")

    print("\nDecoding back to assembly:")
    for code in machine_codes:
```

```
        print(f"{code}  -->  {decode_instruction(code)}")
```

Sample Output:

```
===== PART A: NUMBER SYSTEM CONVERSION =====
Decimal 156  -> Binary : 10011100
Decimal 156  -> Octal  : 234
Decimal 156  -> Hex    : 9C
Hex 9C       -> Decimal: 156
Binary 10011100 -> Hex : 9C

Arithmetic in hex : 1A + 0F = 29
Arithmetic in binary: 1010 - 0011 = 111
Arithmetic in octal: 17 * 3 = 55

===== PART B: INSTRUCTION ENCODE / DECODE =====
MOV R0, #5  -->  000100000101  (hex: 0x105)
ADD R1, #9  -->  001000011001  (hex: 0x219)
SUB R2, #3  -->  001100100011  (hex: 0x323)
LOAD R3, #7  -->  010000110111  (hex: 0x437)

Decoding back to assembly:
000100000101  -->  MOV R0, #5
001000011001  -->  ADD R1, #9
001100100011  -->  SUB R2, #3
010000110111  -->  LOAD R3, #7
```

Code of Conduct:

I G.Saranraj (192511182) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases